

# Type Analysis (1)

CSE552 Program Analysis — Lecture 3

Jaemin Hong

# Type Errors

- ▶ Using non-numbers for arithmetic operations
- ▶ Calling non-functions
- ▶ Providing wrong number of arguments to functions
- ▶ Having field accesses on non-structs
- ▶ Accessing non-existent fields of structs

# Type Analysis

- ▶ Decides whether a program will result in a type error at runtime
- ▶ Sound type analysis
  - ▶ If the analysis concludes that the program is ok, then it will not result in a type error at runtime
  - ▶ Some programs that will not result in a type error at runtime may be classified “not ok” by the analysis (false alarms)

# Type Analysis and Type Checking

- ▶ The terms “type analysis” and “type checking” can be used interchangeably; type checking is one example of static analysis
- ▶ The term “type checking” often refers to the language-default type analysis, as in statically typed languages (e.g., C, Java, Rust)
- ▶ The term “type analysis” often refers to analysis for dynamically typed languages (e.g., Python, JavaScript) or more precise analysis than the default type checking

# Type Analysis and Type Inference

- ▶ In some languages (e.g., C), the types of variables are explicitly annotated by the programmer; type analysis can just utilize this information
- ▶ In many other languages, such type annotations are optional or not available; type analysis needs to decide the types of variables
- ▶ When saying “type checking,” this process is called “type inference”
- ▶ When saying “type analysis,” inferring types is often considered as part of the analysis

# Type Analysis in This Lecture — Scope

- ▶ We target Rust
- ▶ Rust already has type checking
- ▶ Parameters have type annotations (except for anonymous functions)
- ▶ We will consider a subset of Rust with all type annotations removed
- ▶ While this is not practically useful, it allows us to understand the basics of type analysis

# Type Analysis in This Lecture — Approach

- ▶ Our type analysis will collect constraints and solve them using unification, which is a general technique for solving constraints in various analyses (e.g., Steensgaard-style pointer analysis)
- ▶ This type analysis is a simpler version of the Hindley-Milner algorithm, used in real-world languages (e.g., OCaml, Haskell)<sup>12</sup>

---

<sup>1</sup>The principal type-scheme of an object in combinatory logic (Hindley, 1969)

<sup>2</sup>A theory of type polymorphism in programming (Milner, 1978)

# Syntax

Function  $f ::= \text{fn } x(x, \dots, x) \{ s \dots s e \}$

Statement  $s ::= \text{let } x = e; \mid e;$

Expression  $e ::= x \mid n \mid e + e \mid e == e \mid x = e$   
 $\mid \text{if } e \{ e \} \text{ else } \{ e \}$   
 $\mid e(e, \dots, e) \mid \&x \mid *e$

# Types

$$T ::= \text{i32} \mid \text{bool} \mid () \mid \&T \mid \text{fn}(T, \dots, T) \rightarrow T \mid X$$

( $X$  is a type variable)

- ▶ A proper type is a type that is not a type variable (i.e., built from type constructors)
- ▶ Type constructors:  $\text{i32}$ ,  $\text{bool}$ ,  $()$  (nullary),  $\&$  (unary),  $\text{fn}(\_, \dots, \_) \rightarrow \_$  ( $n+1$ -ary)
- ▶ Notation: write a proper type as  $C(T_1, \dots, T_n)$  where  $C$  is the constructor and  $n$  is its arity

## Constraints — Type Variables

- ▶ For each identifier (local variable, parameter, and function name)  $x$ , we introduce a type variable  $\llbracket x \rrbracket$
- ▶ For each occurrence of a non-identifier expression  $e$ , we introduce a type variable  $\llbracket e \rrbracket$
- ▶ Here,  $\llbracket e \rrbracket$  refers to a concrete node in the tree, not to the syntax it corresponds to

## Constraints — Collection Rules (Part 1)

The constraints are collected from each function, statement, and expression in the given program.

- ▶  $\text{fn } x(x_1, \dots, x_n) \{ \dots; e \}: \llbracket x \rrbracket = \text{fn}(\llbracket x_1 \rrbracket, \dots, \llbracket x_n \rrbracket) \rightarrow \llbracket e \rrbracket$
- ▶  $\text{let } x = e: \llbracket x \rrbracket = \llbracket e \rrbracket$
- ▶  $n: \llbracket n \rrbracket = \text{i32}$
- ▶  $e_1 + e_2: \llbracket e_1 + e_2 \rrbracket = \llbracket e_1 \rrbracket = \llbracket e_2 \rrbracket = \text{i32}$
- ▶  $e_1 == e_2: \llbracket e_1 == e_2 \rrbracket = \text{bool} \wedge \llbracket e_1 \rrbracket = \llbracket e_2 \rrbracket = \text{i32}$

## Constraints — Collection Rules (Part 2)

- ▶  $x = e$ :  $\llbracket x = e \rrbracket = () \wedge \llbracket x \rrbracket = \llbracket e \rrbracket$
- ▶  $\text{if } e_1 \{ e_2 \} \text{ else } \{ e_3 \}$ :  
 $\llbracket \text{if } e_1 \{ e_2 \} \text{ else } \{ e_3 \} \rrbracket = \llbracket e_2 \rrbracket = \llbracket e_3 \rrbracket \wedge \llbracket e_1 \rrbracket = \text{bool}$
- ▶  $e(e_1, \dots, e_n)$ :  $\llbracket e \rrbracket = \text{fn}(\llbracket e_1 \rrbracket, \dots, \llbracket e_n \rrbracket) \rightarrow \llbracket e(e_1, \dots, e_n) \rrbracket$
- ▶  $\&x$ :  $\llbracket \&x \rrbracket = \&\llbracket x \rrbracket$
- ▶  $*e$ :  $\&\llbracket *e \rrbracket = \llbracket e \rrbracket$

## Constraints — Solution

- ▶ Solving the constraints gives the type of each identifier and expression
- ▶ If a solution exists, the analysis says “ok”
- ▶ If no solution exists, the analysis says “not ok”

## Example 1

```
fn f(x, y) { x + y }
```

Constraints:

- ▶  $\text{fn } f(x, y) \{x + y\}: \llbracket f \rrbracket = \text{fn}(\llbracket x \rrbracket, \llbracket y \rrbracket) \rightarrow \llbracket x + y \rrbracket$
- ▶  $x + y: \llbracket x + y \rrbracket = \llbracket x \rrbracket + \llbracket y \rrbracket = \text{i32}$

Solution:

- ▶  $\llbracket f \rrbracket = \text{fn}(\text{i32}, \text{i32}) \rightarrow \text{i32}$
- ▶  $\llbracket x \rrbracket = \text{i32}$
- ▶  $\llbracket y \rrbracket = \text{i32}$

## Example 2 — Code and Constraints

```
fn f(x) { x }  
fn g(y) { y(1) }  
fn h() { g(f) }
```

Constraints:

- ▶  $\text{fn } f(x) \{x\}: \llbracket f \rrbracket = \text{fn}(\llbracket x \rrbracket) \rightarrow \llbracket x \rrbracket$
- ▶  $\text{fn } g(y) \{y(1)\}: \llbracket g \rrbracket = \text{fn}(\llbracket y \rrbracket) \rightarrow \llbracket y(1) \rrbracket$
- ▶  $y(1): \llbracket y \rrbracket = \text{fn}(\llbracket 1 \rrbracket) \rightarrow \llbracket y(1) \rrbracket$
- ▶  $1: \llbracket 1 \rrbracket = \text{i32}$
- ▶  $\text{fn } h() \{g(f)\}: \llbracket h \rrbracket = \text{fn}() \rightarrow \llbracket g(f) \rrbracket$
- ▶  $g(f): \llbracket g \rrbracket = \text{fn}(\llbracket f \rrbracket) \rightarrow \llbracket g(f) \rrbracket$

## Example 2 — Solution

```
fn f(x) { x }  
fn g(y) { y(1) }  
fn h() { g(f) }
```

Solution:

- ▶  $\llbracket f \rrbracket = \text{fn}(i32) \rightarrow i32$
- ▶  $\llbracket g \rrbracket = \text{fn}(\text{fn}(i32) \rightarrow i32) \rightarrow i32$
- ▶  $\llbracket y \rrbracket = \text{fn}(i32) \rightarrow i32$
- ▶  $\llbracket h \rrbracket = \text{fn}() \rightarrow i32$

## Example 3

```
fn f(x) { if x { x + 1 } else { 0 } }
```

Constraints:

- ▶  $\text{fn } f(x) \{ \text{if } x \{ x + 1 \} \text{ else } \{ 0 \} \}$ :  
 $\llbracket f \rrbracket = \text{fn}(\llbracket x \rrbracket) \rightarrow \llbracket \text{if } x \{ x + 1 \} \text{ else } \{ 0 \} \rrbracket$
- ▶  $\text{if } x \{ x + 1 \} \text{ else } \{ 0 \}$ :  
 $\llbracket \text{if } x \{ x + 1 \} \text{ else } \{ 0 \} \rrbracket = \llbracket x + 1 \rrbracket = \llbracket 0 \rrbracket \wedge \llbracket x \rrbracket = \text{bool}$
- ▶  $x + 1$ :  $\llbracket x + 1 \rrbracket = \llbracket x \rrbracket = \llbracket 1 \rrbracket = \text{i32}$
- ▶ 1:  $\llbracket 1 \rrbracket = \text{i32}$
- ▶ 0:  $\llbracket 0 \rrbracket = \text{i32}$

No solution exists because  $\llbracket x \rrbracket$  cannot be both `bool` and `i32`.

## Example 4

```
fn f(x) { let y = x + 1; f }
```

Constraints:

- ▶  $\text{fn } f(x) \{ \text{let } y = x + 1; f \}: \llbracket f \rrbracket = \text{fn}(\llbracket x \rrbracket) \rightarrow \llbracket f \rrbracket$
- ▶  $\text{let } y = x + 1: \llbracket y \rrbracket = \llbracket x + 1 \rrbracket$
- ▶  $x + 1: \llbracket x + 1 \rrbracket = \llbracket x \rrbracket + \llbracket 1 \rrbracket = \text{i32}$
- ▶  $1: \llbracket 1 \rrbracket = \text{i32}$

Solution?

- ▶  $\llbracket x \rrbracket = \text{i32}, \llbracket y \rrbracket = \text{i32}, \llbracket f \rrbracket = \text{fn}(\text{i32}) \rightarrow \text{fn}(\text{i32}) \rightarrow \dots$
- ▶  $f$  is a function that can take an integer infinitely many times
- ▶ In a language with a recursive type, we can find a solution:  
 $\mu X. \text{fn}(\text{i32}) \rightarrow X$
- ▶ Our language does not have a recursive type, so no solution exists

## Example 5

```
fn f(x) { x }
```

Constraints:

- ▶  $\text{fn } f(x) \{x\}: \llbracket f \rrbracket = \text{fn}(\llbracket x \rrbracket) \rightarrow \llbracket x \rrbracket$

Solutions:

- ▶  $\llbracket f \rrbracket = \text{fn}(X) \rightarrow X$
- ▶  $\llbracket f \rrbracket = \text{fn}(\text{i32}) \rightarrow \text{i32}$
- ▶  $\llbracket f \rrbracket = \text{fn}(\text{bool}) \rightarrow \text{bool}$
- ▶ ...
- ▶ There are infinitely many solutions
- ▶  $\text{fn}(X) \rightarrow X$  (the one with a type variable) is the most general solution, often called the principal type

# Analogy

Constraints:

$$x + y = 4$$

$$x - y = 2$$

Solution:

$$x = 3, y = 1$$

Constraints:

$$x + y = 4$$

$$2x + 2y = 5$$

No solution

Constraints:

$$x + y = 4$$

$$2x + 2y = 8$$

Infinitely many  
solutions:

$$x = 4 - y$$

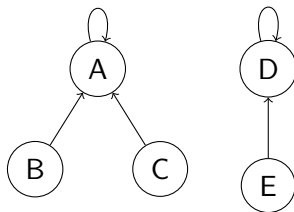
# Union-Find — Introduction

- ▶ Union-find data structure (a.k.a. disjoint-set data structure)<sup>3</sup>
- ▶ Represents and manipulates equivalence relations
- ▶ Consists of a directed graph of nodes that each have exactly one edge to its parent node
  - ▶ Which may be the node itself, in which case it is called a root
- ▶ Two nodes are equivalent if they have a common ancestor, and each root is the canonical representative of its equivalence class

---

<sup>3</sup>An improved equivalence algorithm (Galler and Fischer, 1964)

## Union-Find — Example



# Union-Find — MakeSet

- **MakeSet**( $x$ ): adds a new node  $x$  that initially is its own parent

*MakeSet*( $x$ ):

|  $x.\text{parent} \leftarrow x$

MakeSet(A)



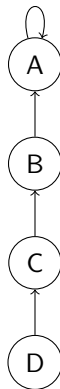
## Union-Find — Find

- ▶ **Find( $x$ )**: finds the canonical representative of  $x$  by traversing the path to the root

*Find*( $x$ ):

```
    while  $x.parent \neq x$  :  
         $x \leftarrow x.parent$   
    return  $x$ 
```

## Union-Find — Find (Example)



$\text{Find}(A)$ ,  $\text{Find}(B)$ ,  $\text{Find}(C)$ ,  $\text{Find}(D)$  all return A

## Union-Find — Union

- ▶ **Union**( $x, y$ ): finds the canonical representatives of  $x$  and  $y$ , and makes one the parent of the other unless they are already equivalent

*Union*( $x, y$ ):

$x_r \leftarrow \text{Find}(x)$

$y_r \leftarrow \text{Find}(y)$

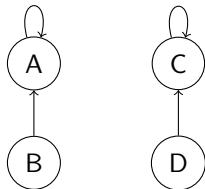
**if**  $x_r \neq y_r$  :

$x_r.\text{parent} \leftarrow y_r$

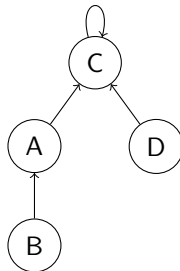
## Union-Find — Union (Example)

Union(B, D)

Before:



After:



# Union-Find — Complexity

- ▶ We can express equivalence between type variables using union-find
- ▶ Using union-find allows us to solve constraints in  $O(n^2)$
- ▶ With path compression and union-by-rank optimizations, the time complexity becomes  $O(n \cdot \alpha(n))$ , where  $\alpha$  is the inverse Ackermann function

# Path Compression

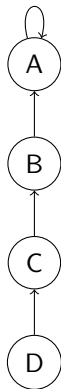
- ▶ During Find, make every node on the path point directly to the root
- ▶ Flattens the tree structure, speeding up future Find operations

*Find(x):*

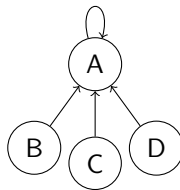
```
| if  $x.parent \neq x$  :  
|    $x.parent \leftarrow \text{Find}(x.parent)$   
| return  $x.parent$ 
```

## Path Compression (Example)

Before Find(D):



After Find(D):



## Union by Rank

- ▶ Each node has a rank (an upper bound on its height)
- ▶ When unioning, attach the smaller-rank tree under the root of the larger-rank tree
- ▶ If ranks are equal, choose one as the new root and increment its rank

*MakeSet*( $x$ ):

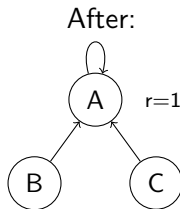
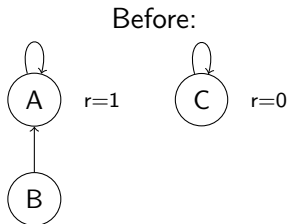
```
|  $x.\text{parent} \leftarrow x$   
|  $x.\text{rank} \leftarrow 0$ 
```

*Union*( $x, y$ ):

```
|  $x_r \leftarrow \text{Find}(x)$   
|  $y_r \leftarrow \text{Find}(y)$   
| if  $x_r = y_r$  :  
|   return  
| if  $x_r.\text{rank} < y_r.\text{rank}$  :  
|    $x_r.\text{parent} \leftarrow y_r$   
| else if  $x_r.\text{rank} > y_r.\text{rank}$  :  
|    $y_r.\text{parent} \leftarrow x_r$   
| else  
|    $y_r.\text{parent} \leftarrow x_r$   
|    $x_r.\text{rank} \leftarrow x_r.\text{rank} + 1$ 
```

## Union by Rank (Example)

Union(B, C): Find(B) = A (rank 1), Find(C) = C (rank 0), so C goes under A



# Summary

- ▶ Type analysis determines whether a program may produce type errors at runtime; a sound analysis guarantees no false negatives
- ▶ Constraint-based type analysis introduces type variables for each identifier and expression, then collects equality constraints from the program's syntax
- ▶ These constraints can be solved using a union-find data structure in almost linear time